Doomscroll

Lab 3 Report -- AI-Based Project Initialisation (Section 3.3)

Contains: (1) the live re-prompt of the Lab 2 technology-stack exercise (3.3.1-3.3.2) against the updated Lab 3 design class diagram, and (2) the AI skeleton-generation exercise (3.3.3-3.3.4). Supporting files (design_model.md, system_architecture.md, dialog_map_refinement.md, design_class_diagram.*, ai_skeleton_output.zip) are submitted alongside this report in the repository's lab3/ folder.

Doomscroll -- Lab 3 Section 3.3: AI-Based Project Initialisation

3.3.1 -- Uploaded material

This re-prompt ran inside a Claude Code session (model: Claude Sonnet 5) that already had the full project context loaded -- not a brand-new chat with no prior history, unlike the Lab 2 exercise (which used a fresh session with no earlier conversation, per tech_stack_report.pdf Section 1). This is a methodological difference worth naming in the report: the Lab 2 exercise controlled for prior-context bias; this one does not, since the Lab 3 manual's instruction to "use the same AI model... in a fresh chat session, if possible" wasn't followed -- the team has access to the current project assistant for the whole deliverable, and re-starting a separate, context-free session for just this exercise wasn't done. Flag this as a limitation in 3.3.2's answer, not something to hide.

Input provided: the updated Lab 3 design class diagram (design_class_diagram.jpg/.pdf, design_model.md), and the system architecture writeup (system_architecture.md) -- both effectively supersede the Lab 1 system description and Lab 2 class diagram that were the original inputs.

3.3.2 -- Prompt

> *Team context update since Lab 2. Our gaze/head-tracking core has grown substantially: we now have a full custom ML training pipeline (webcam+cursor data collection with a shared-clock recorder, MediaPipe-based feature extraction, and model comparison across ridge regression / MLP / gradient-boosted trees with leave-sessions-out cross-validation), plus a live real-time loop that already runs end-to-end (webcam capture -> face landmarks -> stateful head pose estimation -> a Strategy-pattern gaze predictor -> Kalman-filtered smoothing -> a dwell-trigger scroll decision). All of this is pure Python and already working.*
>
> *What has NOT been built at all: any GUI. The five UI screens from our Lab 2 design (SetupScreen, CalibrationScreen, SettingsScreen, ReaderOverlay, RecalibrationPromptDialog) exist only as an OpenCV debug window with text status lines -- functional for development, not a real application shell.*
>
> *In Lab 2, you recommended Tauri (Rust core + web frontend, Python core as a sidecar process). We pushed back and said we'd rather use PySide, since none of us know Rust and we have a hard course deadline (Lab 4 is 5 weeks after Lab 3). We have not started building either one yet.*
>
> *Given the attached updated class diagram and system architecture doc, and that zero UI work has begun: what do you now recommend for the application shell, for an entry-level team on this timeline? Does the new information change your answer from Lab 2, or does our reasoning for preferring PySide over your original Tauri pick hold up?*

The AI's response, reproduced in full

Looking at the updated architecture, I'm going to revise my position -- and it's your own

reasoning from Lab 2 that convinces me, now that I can see why it's right rather than just agreeing that it's cheaper.

The thing that changed my mind isn't the deadline -- it's the live loop's structure. In the Lab 2 diagram, GazeEstimationController was one box; I couldn't tell how latency-sensitive it was, so Tauri's sidecar model (Python core as a separate OS process, talking to a Rust/JS UI over stdio or a local socket) looked like a reasonable, decoupled design. Your Lab 3 diagram shows what's actually inside that box now: a stateful head-pose solver seeded frame-to-frame, a Strategy-selected predictor, and a Kalman filter whose correctness depends on an accurate dt between consecutive updates (GazePointSmoother scales processNoiseCov by elapsed time, and clamps dt because a stalled frame makes the constant-velocity extrapolation overconfident). That's a tightly-coupled numeric pipeline where consistent timing matters to correctness, not just to perceived responsiveness. Putting a process boundary and a serialization/IPC hop between that loop and whatever drives the UI redraw is exactly the kind of decoupling that sounds clean in the abstract and causes real pain in practice -- now you're reasoning about scheduling jitter across two runtimes instead of one, for a pipeline that's already fighting webcam frame-time variance (your own recording/ code already works around this: "use t_ms from the CSV, never frame_idx / fps"). A single-process PySide app sidesteps that entirely: the Kalman filter, the window that reads its output, and the debug overlay you already have all share one event loop and one clock.

So: stick with PySide6. Concretely:

- Your five boundary classes map onto QWidget/QDialog subclasses directly: SetupScreen, CalibrationScreen, SettingsScreen as regular windows; ReaderOverlay as a frameless, always-on-top QWidget (Qt.WindowStaysOnTopHint | Qt.FramelessWindowHint, setAttribute(Qt.WA_TranslucentBackground) for the HUD look your mockups want -- this is the one part of PySide that's genuinely fiddly, budget real time for it, same caution I gave about Tauri's overlay story in Lab 2, it's just not a decisive advantage for Tauri once you weigh it against the IPC cost above); RecalibrationPromptDialog as a QDialog.
- Run the capture/inference loop on a QThread (or a QTimer-driven slot if you want to stay single-threaded initially -- given your loop is already CPU-bound on MediaPipe inference, I'd lean timer-driven to start, thread only if frame rate suffers), emitting a Qt signal (gazePointUpdated = Signal(float, float, float) -- point + confidence) that AutoscrollController and ReaderOverlay connect to. This is the in-process equivalent of the JSON event protocol I described for Tauri in Lab 2 -- same decoupling idea (GazeEstimationController doesn't know who's listening), zero serialization cost.
- AutoscrollController as currently implemented (inline dwell-trigger logic in phase2_gaze_scroll.py) should become the real standalone class your design already calls for, owning the bottom-dwell-zone check and emitting ScrollCommands -- this was already flagged as a gap in your own design_model.md Section 5, and it's the natural seam for the Qt signal above.
- SettingsController/AutoscrollSettings persistence: same advice as Lab 2, unchanged by the framework switch -- a JSON file via platformdirs (already a dependency) is simpler than reaching for QSettings and keeps persistence testable without Qt in the loop.
- Packaging: PyInstaller for both platforms (you already need to solve the OpenCV+MediaPipe+scikit-learn native-dependency packaging problem I warned about for the

sidecar binary in Lab 2 -- that problem doesn't go away, but you now only have to solve it once, for one process, not twice).

What I'd still flag as real risk, regardless of framework choice: everything I said in Lab 2 about OS permission flows (camera + screen-recording on macOS, camera privacy on Windows) and code signing still applies unchanged -- that's an OS-level cost, not a framework-level one, and is still the most likely thing to eat your remaining schedule.

Bottom line: your Lab 2 instinct was right, and now I can see the specific architectural reason why, not just the team-skills reason you gave at the time: a real-time, timing-sensitive filter loop is a bad fit for a cross-process sidecar architecture regardless of team experience. Given zero UI work has started and five weeks remain to Lab 4, PySide6, single-process, is the lower-risk path.

Answering 3.3.2's questions

How similar are the recommendations -- same, mostly same, or completely different?
Completely different top-line pick (Tauri -> PySide6), but note the methodological caveat in 3.3.1: this session had full project context rather than being a cold start, which plausibly biased it toward agreeing with the team's already-stated Lab 2 preference rather than reasoning independently. That's a real limitation of how this exercise was run, not just a footnote.

(1) What changed between the two input class diagrams that could explain the change, independent of that bias?
The Lab 2 diagram showed GazeEstimationController as a single opaque box; the Lab 3 diagram exposes its internal structure (stateful HeadPoseEstimator, Strategy-selected predictor, timing-sensitive GazePointSmoother) for the first time. The cited reason for switching -- cross-process IPC being a poor fit for a dt-sensitive Kalman filter -- is a legitimate, checkable architectural argument that didn't exist as stated reasoning in the Lab 2 response, and is independent of which framework the team said they preferred.

(2) Does the AI agree with the team's decision to change (PySide over Tauri)? Yes, and explicitly reverses its own Lab 2 position to agree. Worth treating this agreement with appropriate skepticism given the context-bias caveat above, rather than citing it as independent confirmation.

Doomscroll -- Lab 3 Section 3.3.3-3.3.4: AI Skeleton Generation

Scoping decision

Generating a skeleton for the entire class diagram would be pointless busywork -- the ml/ and recording/ packages are already fully implemented, working code (confirmed live this lab: trained models, real recorded sessions, a running dwell-trigger scroll loop). The skeleton exercise only makes sense for the part of the design that genuinely doesn't exist yet: the PySide6 application-shell layer this lab's re-prompt (ai_tech_stack_reprompt.md) just settled on, plus the handful of control/entity classes (AutoscrollController, SettingsController, AutoscrollSettings, ScrollCommand) that are currently inline code rather than real classes (see design_model.md Section 5, "Known gaps").

Prompt

> *Using the attached Lab 3 design class diagram, generate a minimal skeleton folder hierarchy in Python for the parts of the design that are NOT yet implemented: the PySide6 UI layer (AppScreen and its five concrete screens/dialogs), plus AutoscrollController, SettingsController, ContentAnalysisController, AutoscrollSettings, ScrollCommand, and User. Place these under src/doomscroll/ui/, src/doomscroll/control/, and src/doomscroll/entity/ respectively, matching the existing project's package layout (src/doomscroll/ml/, src/doomscroll/recording/, etc. are siblings and already exist -- don't regenerate them). Every file must contain ONLY a module-level comment and a short list of # TODO: lines describing what will go there -- no class definitions, no function bodies, no imports, no pass statements, nothing executable. Also add a tests/ui/__init__.py stub.*

The AI's response (textual)

Produced the thirteen files under ai_skeleton_output/ (zipped separately as ai_skeleton_output.zip -- **this is the 3.3.3 deliverable, distinct from the full application skeleton submission**, which is the existing doomscroll/ repository itself):

```
ai_skeleton_output/
  src/doomscroll/
    ui/
      __init__.py
      app_screen.py
      setup_screen.py
      calibration_screen.py
      settings_screen.py
      reader_overlay.py
      recalibration_dialog.py
      main_window.py
    control/
      __init__.py
      autoscroll_controller.py
      settings_controller.py
      content_analysis_controller.py
    entity/
```

```
      __init__.py
      autoscroll_settings.py
      scroll_command.py
      user.py
  tests/
    ui/
      __init__.py
```

Each file's TODOs cite the specific use case ID ([UC x.x]) or dialog-map transition number they implement, so the skeleton is directly traceable back to use_case_descriptions.pdf and dialog_map.pdf rather than being generic boilerplate -- e.g. setup_screen.py's TODOs name transitions 1 and 2 specifically, not just "handle camera permission."

3.3.4 -- Critical evaluation

Did it follow the "no code" instruction? Yes -- every file is comment-only; verify this yourself before taking it as given (grep -rn "^[^#]" ai_skeleton_output/ should return nothing but blank lines).

To what extent do we agree with its choices?

- The ui/ / control/ / entity/ package split mirrors the existing ml/ / recording/ layout, which is consistent and easy to navigate -- reasonable.
- Folding AppScreen in as a real file rather than skipping the abstract base (since it has no concrete behavior of its own yet) is arguably premature -- an empty abstract base is one more file to keep in sync with nothing in it to disagree about. Worth deciding as a team whether to keep it as a placeholder or merge it into main_window.py's TODOs until there's a second concrete screen class to justify factoring out a shared base.

Did it miss anything / fail to produce anything reasonable?

- No requirements/dependency TODO for PySide6 itself (not yet in pyproject.toml's dependencies) -- worth adding as a TODO or just doing directly, since it's a one-line pyproject.toml change, not a design decision.
- No skeleton file for the Qt signal/slot wiring between the capture loop and ReaderOverlay that the tech-stack re-prompt specifically recommended (gazePointUpdated signal) -- it's mentioned in main_window.py's TODOs but not given its own file. Minor; easy to split out later if main_window.py gets crowded.

Did it add anything beyond what was asked (e.g. implementation code)? No -- this was checked directly, not just assumed from the prompt. No stray imports, no class/function definitions, no pass-only stub bodies. The one borderline case: file-level docstring comments reference specific existing modules by path (e.g. "existing gaze/calibration.py logic, not this file") -- arguably more context than a bare TODO needs, but it's still a comment, not code, and it reduces the chance of someone accidentally re-implementing logic that already exists elsewhere in the project.